

[← writing](#)

Harness engineering: why agent performance now lives outside the model

June 10, 2026 · 10 min read

TL ; DR

- An agent is a model plus a harness: the orchestration code, prompts, tools, memory, and verification around the weights. The harness now drives more performance variation than the model.
- Tsinghua's Natural-Language Agent Harness makes that logic explicit and ablatable. Rewriting one harness from code into natural language moved a benchmark 16.8 points with the same model and strategy.
- Stanford's Meta-Harness optimizes the harness automatically. Its top configuration put a smaller model above a larger one, and a harness tuned on one model transferred gains to five others.
- The recurring lesson is subtraction: most added structure hurt. The only module that consistently helped was the one that narrowed the agent's attempt loop.

Here's a situation you may recognize. You've built an agent, you swap in the newest model expecting a jump, and the numbers barely move. Meanwhile some other team, running the exact same model on the exact same benchmark, is scoring several times higher than you. If the model is identical, where is that gap coming from?

The evidence now points at everything around the model. Stanford researchers measured more performance variation coming from orchestration code than from the choice of model. LangChain rebuilt only the infrastructure around their coding agent, touched no model, and jumped from outside the top 30 to rank five on Terminal-Bench 2. On the same model, the gap between a good harness and a bad one can reach six times.

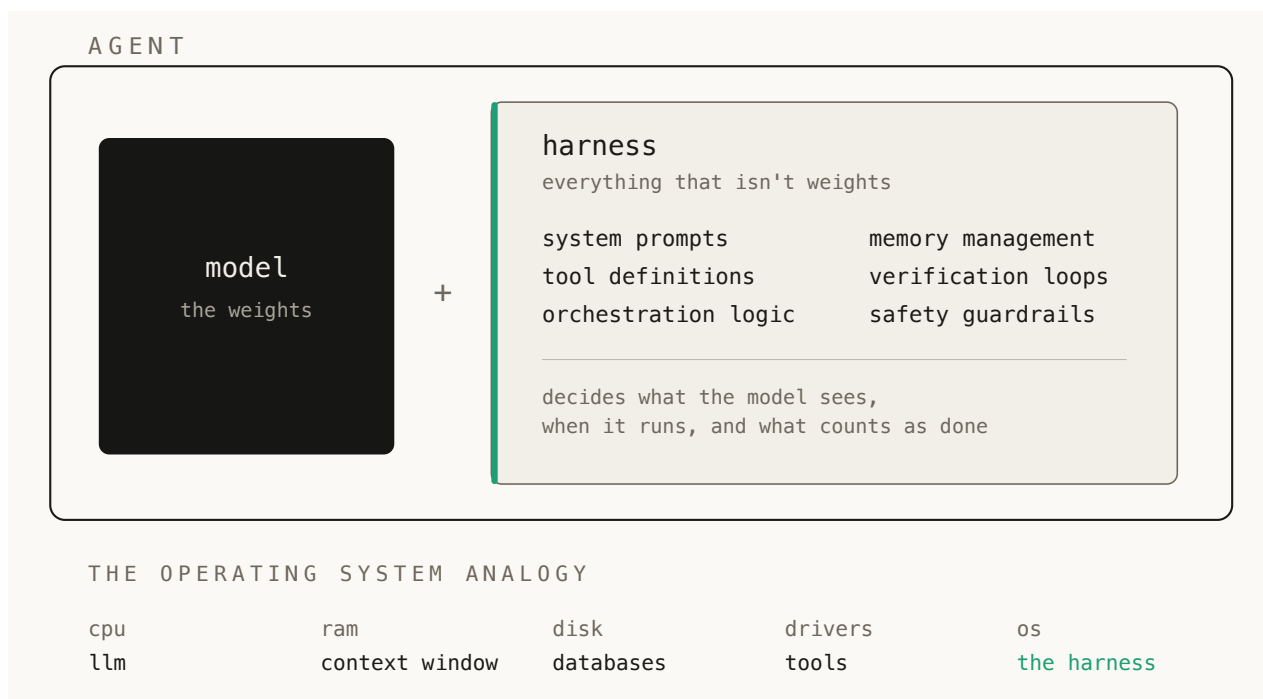
So the interesting question stops being "which model" and becomes "what is this surrounding layer, and can we engineer it deliberately?" In March 2026, two papers turned that question into a discipline: Natural-Language Agent Harnesses from Tsinghua and

Meta-Harness from Stanford. I covered what a harness is, and the nine components of a good one, in a companion essay. This one follows the evidence: what happens when researchers make the harness explicit, measure it, and let machines optimize it. And I'll flag one twist now so you can watch for it: when the measuring finally happened, most of the structure people had been adding turned out to hurt. We'll get to which module survived.

An agent is a model plus a harness

Before we can measure the harness, we need to agree on what it is. LangChain frames it with the sharpest line in the field: if you're not the model, you're the harness.

If that still feels abstract, an operating system is the right anchor. The LLM plays the CPU. The context window is RAM, fast but scarce. External databases stand in for disk. Tool integrations are the device drivers. And the harness is the operating system: it schedules the processor, decides what reaches it, and turns raw cycles into finished work. The weights do the thinking, but the harness decides what there is to think about.



The anatomy of an agent. The weights are the smallest part of the system.

Concretely, the harness is everything that isn't model weights: system prompts, tool definitions, orchestration logic, memory management, verification loops, safety guardrails. Anthropic identified five canonical patterns for it: prompt chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer loops. Each is a different strategy for when and how the model gets called, and every production agent combines them.

Those architectural choices are where the six-times gaps live, not the weights underneath. Which raises an awkward question: if this layer matters so much, why hasn't anyone been engineering it carefully all along?

Why nobody could measure it

The honest answer is that until recently, nobody could even see the harness clearly. It gets built by accretion.

Harness logic ends up scattered across controller code, framework defaults, and verifier scripts, so no one can point to it as a single object. The Tsinghua authors note that two systems which nominally differed by one design choice actually differed in prompts, tools, verification gates, and state semantics simultaneously. You can't run a controlled experiment on a variable you can't isolate, so nobody did.

Anthropic's own evolution shows what building blind looks like. Naive harnesses suffer two failure modes: one-shotting, where the agent tries everything at once and exhausts its context, and premature completion, where a later session sees partial progress and declares victory. Their fix evolved into a three-agent, GAN-inspired architecture: a planner, a generator, and an evaluator that clicks through the running app like a real user. It was twenty times more expensive, roughly \$200 per run instead of \$9. But the system finally worked end to end. Notice the shape of that story: they got to a working harness by trial and error, not by measurement.

OpenAI converged on the same place independently. One internal effort produced a million lines of application logic, tests, CI, and tooling in five months with zero lines written by hand, and the team's discovery was that their primary job had become enabling agents to do useful work. Productive, but ad hoc, non-portable, and impossible to ablate.

Standards did start to emerge around the edges. AGENTS.md reached 60,000 repositories, and Anthropic's agent skills added reusable procedures. But both package components, conventions, and snippets, not the full harness itself. What the field needed was harness logic made explicit and executable, something you could hold constant, swap out, and measure. That is exactly what the first paper builds.

What if the harness were written in natural language?

The Tsinghua team asks a simple question: what if you could write an agent's entire control logic, not in Python, not in YAML, but in structured natural language?

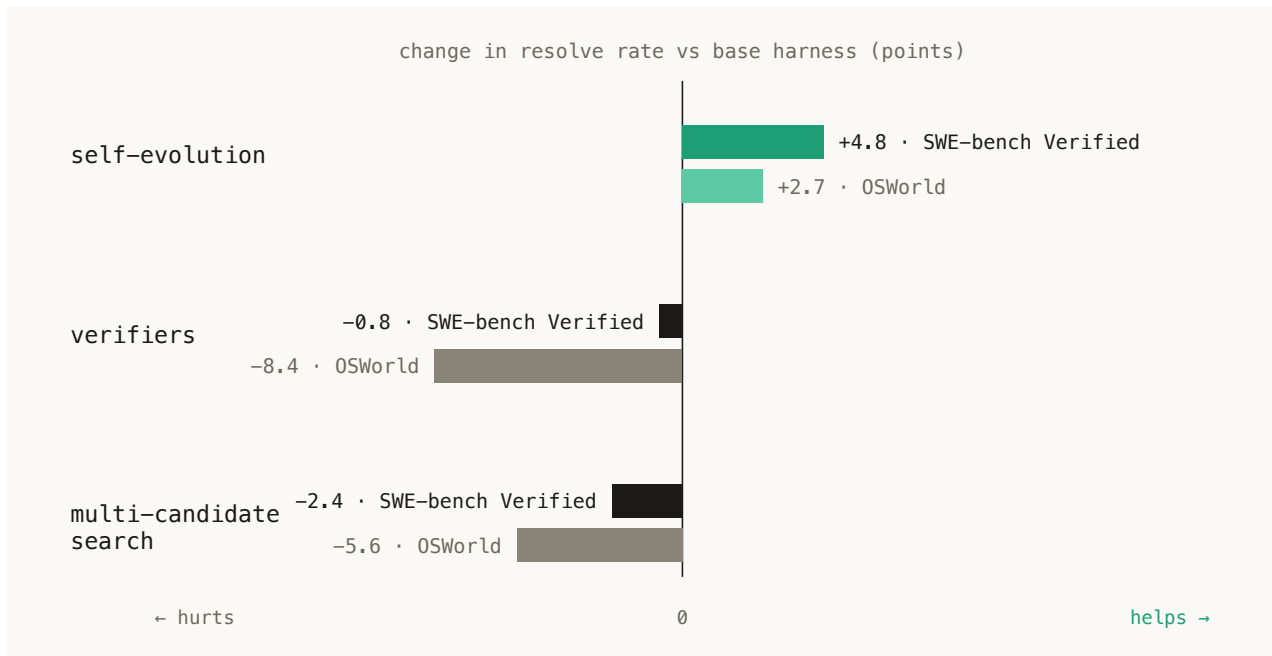
Their Natural-Language Agent Harness (NLAH) separates an agent into three layers: the backend (infrastructure and tools), a runtime charter (the universal physics: how contracts bind, how state persists, how child agents are managed), and the NLAH itself (task-specific control logic: contracts, roles, state structure, failure taxonomies).

That separation is the whole point, because it fixes the measurement problem from the last section. Swap the NLAH while fixing the charter and you're testing harness design. Fix the NLAH while swapping the charter and you're testing runtime policy. Clean ablation, at last.

Two mechanisms make the natural-language version trustworthy enough to run. Execution contracts turn fuzzy LLM completions into bounded agent calls with five elements: required inputs, budgets, permissions, completion conditions, and output paths. A contract does for an agent call what a function signature does for code, which is to say it pins down what goes in and what must come out. And file-backed state externalizes memory to path-addressable files, plain files on disk that survive truncation, restarts, and delegation.

So does all this structure help? Here's the first surprise. On SWE-bench Verified with GPT-5.4 at maximum reasoning, resolve rates clustered between 74% and 76% regardless of configuration. But the full harness burned 16.3 million prompt tokens per sample across 642 tool calls and 32 minutes, and the stripped-down version got there in 1.2 million tokens, 51 calls, under 7 minutes. Same destination, fourteen times the compute. The extra machinery wasn't buying accuracy, it was buying overhead.

Then the module-by-module ablation, the twist I promised at the start, found something stranger.



Module ablations from the NLAH paper. More structure is not always better.

Self-evolution was the only consistently helpful module: plus 4.8 points on SWE-bench Verified and plus 2.7 on OSWorld, via an acceptance-gated attempt loop that stays narrow until failure signals justify broadening. Verifiers, the module everyone's instinct says to add, hurt: minus 0.8 and minus 8.4. Multi-candidate search hurt too: minus 2.4 and minus 5.6. So the one survivor is the module that narrows the agent's attempt loop, and everything that broadened the search made things worse.

The headline finding came from a different experiment. The researchers took OS-Symphony, a native code harness for desktop automation, and migrated its logic into NLAH representation. Same strategy, same model, different representation. Performance jumped from 30.4% to 47.2%. Runtime dropped from 361 minutes to 141. LLM calls collapsed from 1,200 to 34. The representation itself drove the gain, replacing brittle GUI repair loops with durable runtime state and artifact-backed completion.

Step back and two patterns crystallize from the full results. Roughly 90% of all compute flows through delegated child agents, not the parent, which means the harness is an orchestration pattern rather than a reasoning pattern: it decomposes, delegates, and verifies. And in every configuration tested, narrowing the loop beat broadening the search. Now, if changing the representation alone can move a benchmark 16.8 points, an obvious next question appears: can a machine find the right harness for you?

Letting the harness optimize itself

That's the question Meta-Harness answers. It comes from a Stanford team including Omar Khattab, the creator of DSPy, and it treats the harness as an optimization target. DSPy tunes prompts within a fixed pipeline. Meta-Harness rewrites the pipeline itself: structure, retrieval, memory, orchestration topology.

The loop works like this. An agentic proposer (Claude Code running Opus 4.6) reads the failed execution traces of prior candidates, diagnoses what broke, and writes a complete new harness. Scores and raw traces accumulate in a growing file system. An evaluator tests each proposal. Repeat.

The scale is striking: about 10 million tokens per iteration, 400 times more feedback than prior text-optimization methods, 82 files read per round. And those traces turn out to be irreplaceable. Remove them and accuracy drops from 50% to 34.6%. Replace them with summaries and you get 34.9%, barely better than nothing. The signal lives in the raw traces; compression destroys it.

Now for what this buys you, because it changes the calculus of model choice. On Terminal-Bench 2, Meta-Harness scored 76.4%, the only automatically optimized system in a field of hand-engineered entries. It reached rank two with Opus and rank one with Haiku: a smaller model outranking larger ones through harness optimization alone. On a 215-class text classification task it hit 48.6% accuracy, 7.7 points above the prior state of the art, using four times fewer tokens. And a harness optimized on one model transferred to five others, improving all of them.

So the asset that transfers, the thing worth investing in, is not the model. It's the harness. Two more systems fill in the edges of that picture.

Constraints and safety complete the picture

DeepMind's AutoHarness compiles game rules into code harnesses, eliminating 100% of illegal moves across 145 games; one variant replaces the LLM entirely and runs the decision policy as pure code. And AgentSpec provides safety constraints as a domain-specific language, preventing over 90% of unsafe executions in evaluation.

Put the four systems side by side and they read as facets of one discipline: representation, optimization, constraints, safety. Which is worth situating in a longer arc, because we've been here before.

Three eras, and a craft of subtraction

Prompt engineering gave way to context engineering, which is now giving way to harness engineering. Three eras in four years, each absorbing the one before it: harness engineering contains the prior two and adds what the model can't do on its own, from orchestration and memory through verification and safety.



Each era absorbs the one before it.

In practice the discipline takes on an odd shape, and it rhymes with the ablation results. Anthropic named the dynamic: every harness component encodes an assumption about what the model can't do alone, and those assumptions expire. When Opus 4.6 stopped needing context resets, Anthropic dropped them entirely. Manus rewrote their harness five times in six months. Vercel removed 80% of an agent's tools and got better results.

The harness space doesn't shrink as models improve. It moves. Which is why mature harness work looks less like adding structure and more like deleting it on schedule, as the assumptions underneath expire. That's the practice; here's what it means for you.

If you build agents, you are a harness engineer

Back to the gap we started with, the two teams on the same model with a six-times difference between them. The takeaway is unambiguous: investing in your harness yields larger, faster, and more reliable gains than waiting for the next model upgrade. It's no longer a question of which model to pick. It's a question of which structure to remove.

Open problems remain, and they're not small. Portable harness logic lowers the barrier to spreading risky workflows: prompt injection buried in harness text, malicious tools grafted

into shared artifacts. One audit found that one in four community-contributed agent skills contains a vulnerability. And the most consequential open question sits further out: can harness and model weights be co-evolved, letting strategy shape what the model learns and the model reshape the strategy that wraps it?

Harness engineering is moving from folklore to measurement. The layer between a language model and useful work always carried more weight than it got credit for. Now it has a name, two papers, and a benchmark trail. And like any other system, it rewards careful engineering.

Sources

- [Natural-Language Agent Harnesses](#), Pan et al., Tsinghua University, March 2026
- [Meta-Harness: End-to-End Optimization of Model Harnesses](#), Lee, Nair, Zhang, Lee, Khattab, and Finn, March 2026 ([code](#))
- [Building effective agents](#), Anthropic
- [AGENTS.md](#), the emerging convention for agent-readable project instructions
- PY's video survey [Rethinking AI Agents: The Rise of Harness Engineering](#), which first connected these papers for me



Muhammad Farooq

I research, build, and teach AI systems.

[More writing](#) →

[AI consulting](#) →